

```
apue$ mkdir ~/cuoi_ky
```

```
apue$ cd ~/cuoi_ky
```

//tạo file file_manager.h

```
apue$ cat > file_manager.h
```

```
#ifndef FILE_MANAGER_H // Bảo vệ không include 2 lần
```

```
#define FILE_MANAGER_H
```

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <unistd.h>
```

```
#include <time.h>
```

```
//định nghĩa kích thước
```

```
#define MAX_FILENAME 256
```

```
#define MAX_USERNAME 32
```

```
#define MAX_GROUPNAME 32
```

```
#define MAX_ERROR_MSG 128
```

```
// Cấu trúc lưu trữ thông tin file
```

```
struct file_info {
```

```
    char filename[MAX_FILENAME]; // Tên file
```

```
    mode_t mode;                // Loại file và permissions
```

```
    nlink_t nlink;              // Số lượng links
```

```
    off_t size;                  // Kích thước file
```

```
    ino_t inode;                 // Số inode
```

```
    dev_t device;                // ID thiết bị
```

```

time_t atime;          // Thời gian truy cập cuối
time_t mtime;          // Thời gian sửa đổi cuối
time_t ctime;          // Thời gian thay đổi metadata
uid_t uid;             // User ID
gid_t gid;             // ilename, struct file_info *info);

int get_user_group_info(struct file_info *info);
void determine_file_type(mode_t mode, char *type_str);
void convert_permissions(mode_t mode, char *perm_str);

#endif

```

//tạo file file_info_collector.c

```
cat > file_info_collector.c
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <pwd.h>
#include <grp.h>
#include <time.h>
#include <errno.h>
#include <unistd.h>
#include "file_manager.h"

```

```
// Thu thập thông tin file từ hệ thống
```

```

int collect_file_stats(const char *filename, struct file_info *info) {
    struct stat file_stat; // lưu info từ hệ thống

    // Gọi hàm lstat() để lấy thông tin file 0 true | -1 false
    if (lstat(filename, &file_stat) == -1) {
        // ghi thông báo lỗi vào struct
        snprintf(info->error_msg, sizeof(info->error_msg),
            "Error: %s", strerror(errno));
        return -1; // Trả về -1 để báo lỗi
    }

    // Copy thông tin từ struct stat vào struct file_info
    strncpy(info->filename, filename, sizeof(info->filename)-1);
    info->mode = file_stat.st_mode; // Loại file và permissions
    info->nlink = file_stat.st_nlink; // Số lượng links
    info->size = file_stat.st_size; // Kích thước file (bytes)
    info->inode = file_stat.st_ino; // Số inode
    info->device = file_stat.st_dev; // ID thiết bị
    info->atime = file_stat.st_atime; // Thời gian truy cập cuối
    info->mtime = file_stat.st_mtime; // Thời gian sửa đổi cuối
    info->ctime = file_stat.st_ctime; // Thời gian thay đổi metadata
    info->uid = file_stat.st_uid; // User ID
    info->gid = file_stat.st_gid; // Group ID

    return 0;
}

```

//Lấy username và groupname từ ID

```

int get_user_group_info(struct file_info *info) {
    struct passwd *pwd = getpwuid(info->uid); //tìm tên uid

```

```

if (pwd) {
    // Chuyển uid thành username
    strncpy(info->username, pwd->pw_name, sizeof(info->username)-1);
} else {
    // không tìm thấy user, dùng số ID
    snprintf(info->username, sizeof(info->username), "%d", info->uid);
}

struct group *grp = getgrgid(info->gid);    //tìm tên gid
if (grp) {
    // Chuyển gid thành groupname
    strncpy(info->groupname, grp->gr_name, sizeof(info->groupname)-1);
} else {
    snprintf(info->groupname, sizeof(info->groupname), "%d", info->gid);
}

return 0;
}

// Xác định loại file
void determine_file_type(mode_t mode, char *type_str) {

    //các hàm kiểm tra
    if (S_ISREG(mode)) // .txt, .docx, .jpg, .png, .c, .py, .exe
        strcpy(type_str, "Regular File");

    else if (S_ISDIR(mode)) //folder
        strcpy(type_str, "Directory");

    else if (S_ISLNK(mode)) //chứa link

```

```

        strcpy(type_str, "Symbolic Link");

    else if (S_ISCHR(mode))
        strcpy(type_str, "Character Device");

    else if (S_ISBLK(mode))//các thiết bị đọc ghi vd Ổ đĩa cứng (Hard drive), USB drive, CD-ROM.
        strcpy(type_str, "Block Device");

    else if (S_ISFIFO(mode)) //(First-In, First-Out / Ống dẫn)
        strcpy(type_str, "FIFO/Pipe");

    else if (S_ISSOCK(mode))
        strcpy(type_str, "Socket");
    else
        strcpy(type_str, "Unknown Type");
}

```

//Chuyển permissions sang dạng "rwxrwxrwx"

```

void convert_permissions(mode_t mode, char *perm_str) {
    // User permissions (chủ sở hữu)
    perm_str[0] = (mode & S_IRUSR) ? 'r' : '-'; // Read permission
    perm_str[1] = (mode & S_IWUSR) ? 'w' : '-'; // Write permission
    perm_str[2] = (mode & S_IXUSR) ? 'x' : '-'; // Execute permission

    // Group permissions (nhóm)
    perm_str[3] = (mode & S_IRGRP) ? 'r' : '-'; // Read permission
    perm_str[4] = (mode & S_IWGRP) ? 'w' : '-'; // Write permission
    perm_str[5] = (mode & S_IXGRP) ? 'x' : '-'; // Execute permission
}

```

```

// Others permissions (người khác)
perm_str[6] = (mode & S_IROTH) ? 'r' : '-'; // Read permission
perm_str[7] = (mode & S_IWOTH) ? 'w' : '-'; // Write permission
perm_str[8] = (mode & S_IXOTH) ? 'x' : '-'; // Execute permission

perm_str[9] = '\0';
}

```

//tạo file fileinfo.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <glob.h>
#include <sys/stat.h>
#include <time.h>
#include <pwd.h>
#include <grp.h>
#include <unistd.h>

#include "file_manager.h"

/* ===== COLORS & ICONS ===== */
#define RESET "\033[0m"
#define RED "\033[31m" // Màu đỏ
#define GREEN "\033[32m" // Màu xanh lá
#define YELLOW "\033[33m" // Màu vàng
#define BLUE "\033[34m" // Màu xanh dương
#define MAGENTA "\033[35m" // Màu tím

```

```

#define CYAN    "\033[36m"    // Màu xanh lơ
#define BOLD    "\033[1m"      // In đậm

// Hàm lấy Icon dựa trên tên file
const char* get_file_icon(const char *filename, mode_t mode) {

    if (S_ISDIR(mode)) return "📁";

    if (S_ISLNK(mode)) return "🔗";

    const char *ext = strrchr(filename, '.');

    if (!ext) return "📄"; // Mặc định

    if (strcmp(ext, ".c") == 0) return "c ";
    if (strcmp(ext, ".h") == 0) return "🔑 ";
    if (strcmp(ext, ".txt") == 0) return "📄 ";
    if (strcmp(ext, ".pdf") == 0) return "📄 ";
    if (strcmp(ext, ".jpg") == 0 || strcmp(ext, ".png") == 0) return "🖼️ ";
    if (strcmp(ext, ".exe") == 0 || strcmp(ext, ".sh") == 0) return "🚀 ";
    if (strcmp(ext, ".zip") == 0 || strcmp(ext, ".tar") == 0) return "📦 ";

    return "📄 ";
}

/* ===== WILDCARD EXPAND ===== */

char **expand_pattern(const char *pattern, int *count) {

    glob_t results;

    *count = 0;

```

```

if (glob(pattern, 0, NULL, &results) != 0)
    return NULL;

*count = results.gl_pathc;
if (*count == 0)
    return NULL;

char **files = malloc(sizeof(char*) * (*count));
for (size_t i = 0; i < results.gl_pathc; i++) {
    files[i] = strdup(results.gl_pathv[i]);
}

globfree(&results);
return files;
}

int file_exists(const char *path) {
    struct stat s;
    return (lstat(path, &s) == 0);
}

/* Thu thập các file từ argv */
char **collect_files(int argc, char *argv[], int *count) {
    char **result = NULL;
    int total = 0;

    for (int i = 1; i < argc; i++) {
        char *arg = argv[i];

        if (strchr(arg, '*') || strchr(arg, '?')) {

```



```

    int n = 0;

    char **expanded = expand_pattern(arg, &n);

    if (expanded) {
        result = realloc(result, sizeof(char*) * (total + n));

        for (int j = 0; j < n; j++)
            result[total + j] = expanded[j];

        total += n;

        free(expanded);
    }
} else {
    result = realloc(result, sizeof(char*) * (total + 1));

    result[total++] = strdup(arg);
}
}

*count = total;

return result;
}

/* ===== FORMAT TIME ===== */

char *format_time(time_t t) {
    static char buf[64];

    struct tm lt;

    localtime_r(&t, &lt);

    strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", &lt);

    return buf;
}

```

```
/* ===== FORMAT SIZE ===== */
```

```
char *human_size(off_t bytes) {  
    static char buf[64];  
    double size = (double)bytes;  
  
    if (size < 1024) {  
        snprintf(buf, sizeof(buf), "%ld bytes", (long)bytes);  
    }  
    else if (size < 1024 * 1024) {  
        snprintf(buf, sizeof(buf), "%.1f KB", size / 1024.0);  
    }  
    else if (size < 1024 * 1024 * 1024) {  
        snprintf(buf, sizeof(buf), "%.1f MB", size / (1024.0 * 1024.0));  
    }  
    else {  
        snprintf(buf, sizeof(buf), "%.2f GB", size / (1024.0 * 1024.0 * 1024.0));  
    }  
    return buf;  
}
```

```
void print_colored_perms(const char *perm_str) {  
    for (int i = 0; i < 9; i++) {  
        if (perm_str[i] == 'r') printf("%s%c", YELLOW, perm_str[i]);  
        else if (perm_str[i] == 'w') printf("%s%c", RED, perm_str[i]);  
        else if (perm_str[i] == 'x') printf("%s%c", GREEN, perm_str[i]);  
        else printf("%s%c", RESET, perm_str[i]); // Dấu gạch ngang  
    }  
    printf("%s", RESET); // Reset màu  
}
```

```
/* ===== PRINT FILE INFO ===== */
```

```
void print_file_info(struct file_info *info) {  
    char type_str[50];  
    char perm_str[10];  
  
    determine_file_type(info->mode, type_str);  
    convert_permissions(info->mode, perm_str);  
  
    // Chọn màu tiêu đề dựa trên loại file  
    const char *title_color = GREEN;  
    if (S_ISDIR(info->mode)) title_color = BLUE;  
    if (S_ISLNK(info->mode)) title_color = CYAN;  
    if (info->mode & S_IXUSR) title_color = MAGENTA; // File thực thi  
  
    // Lấy icon xịn  
    const char *icon = get_file_icon(info->filename, info->mode);  
  
    printf("\n%s%s %s%s%s\n", title_color, icon, BOLD, info->filename, RESET);  
  
    printf(" ┆ %sType:%s %s", BOLD, RESET, type_str);  
    printf(" ("); print_colored_perms(perm_str); printf(")\n");  
  
    // Tô màu size: Nếu > 1MB thì màu đỏ cảnh báo, ngược lại màu xanh  
    const char *size_color = (info->size > 1024*1024) ? RED : GREEN;  
    printf(" ┆ %sSize:%s %s%s (%ld bytes)\n",  
        BOLD, RESET, size_color, human_size(info->size), RESET, (long)info->size);  
  
    printf(" ┆ %sOwner:%s %s%s:%s%s\n",
```

```
BOLD, RESET, CYAN, info->username, info->groupname, RESET);
```

```
printf(" |— %sLinks:%s %ld %sInode:%s %ld\n",
```

BOLD, RESET, (long)info->nlink, BOLD, RESET, (long)info->inode);

```
printf(" ┃ %sTimes:%s\n", BOLD, RESET);
```

```
printf(" | ─ Access: %s%s%s\n", YELLOW, format_time(info->atime), RESET);
```

```
printf(" | — Modify: %s%s%s\n", BLUE, format_time(info->mtime), RESET);
```

```
printf(" |  └─ Change: %s\n", format_time(info->ctime));
```

```
printf("  └─ %sDevice:%s %ld\n", BOLD, RESET, (long)info->device);
```

```
// Thêm đường kẻ mờ để ngăn cách các file
```

```
printf("-----\n");
```

$$\}$$

```
/*=====PROCESS FILE=====*/
```

```
void process_file(const char *filename) {
```

```
struct file_info info;
```

```
if (collect_file_stats(filename, &info) != 0) {
```

```
printf("❌ Lỗi: %s → %s\n", filename, info.error_msg);
```

return;

$$\}$$

```
get_user_group_info(&info);
```

```
print file info(&info);
```

$$\}$$

```
/* ===== SORT STRINGS ===== */
```

```
int cmp_str(const void *a, const void *b) {  
    char * const *sa = a;  
    char * const *sb = b;  
    return strcmp(*sa, *sb);  
}
```

```
/* ===== MAIN ===== */
```

```
int main(int argc, char *argv[]) {  
  
    if (argc < 2) {  
        printf("Cách dùng: ./fileinfo <file1> <file2> *.txt ...\n");  
        return 1;  
    }
```

```
    int count = 0;  
    char **list = collect_files(argc, argv, &count);
```

```
    if (!list || count == 0) {  
        printf("Không có file nào để xử lý.\n");  
        return 0;  
    }
```

```
    char **valid = NULL;
```

```
    int valid_count = 0;
```

```
    for (int i = 0; i < count; i++) {  
        if (file_exists(list[i])) {
```

```

        valid = realloc(valid, sizeof(char*) * (valid_count + 1));
        valid[valid_count++] = list[i];
    } else {
        printf("⚠ Warning: File không tồn tại → %s\n", list[i]);
        free(list[i]);
    }
}

free(list);

if (valid_count == 0) {
    printf("Không có file hợp lệ.\n");
    return 0;
}

qsort(valid, valid_count, sizeof(char*), cmp_str);

for (int i = 0; i < valid_count; i++) {
    printf("🔄 Đang xử lý file %d/%d: %s\n", i + 1, valid_count, valid[i]);
    process_file(valid[i]);
    free(valid[i]);
}

free(valid);

printf("\n✅ Đã xử lý xong %d file.\n", valid_count);
return 0;
}

```

//tạo makefile

CC = gcc

CFLAGS = -Wall -Wextra -g

TARGET = fileinfo

SOURCES = file_info_collector.c fileinfo.c

OBJECTS = \$(SOURCES:.c=.o)

all: \$(TARGET)

\$(TARGET): \$(OBJECTS)

\$(CC) \$(CFLAGS) -o \$(TARGET) \$(OBJECTS)

clean:

rm -f \$(TARGET) \$(OBJECTS)

//biên dịch

make

//tạo folder test

apue\$ mkdir test_file

cd test_file

//tạo các loại file test

apue\$ echo "hello" > a.txt

apue\$ echo "int main(){}" > b.c

```
apue$ touch empty.txt
```

```
apue$ echo "binary" > run.sh
```

```
apue$ mkdir myfolder
```

```
apue$ ln -s a.txt link_to_a
```

```
apue$ ln -s myfolder link_to_folder
```

```
apue$ mkfifo mypipe
```

```
apue$ nc -lU mysocket &
```

(thiếu 3 file hệ thống vì mình sẽ lấy từ hệ thống không tạo được)

//lui về cuoi_ky

```
apue$ cd ../
```

//chạy

```
apue$ ./fileinfo test_file/*
```

(kết quả ở đây)

Test riêng các file hệ thống

```
./fileinfo /dev/wd0
```

```
./fileinfo /dev/wd0a
```

```
./fileinfo /dev/tty
```

```
./fileinfo /dev/null
```